

ROS2 Controllers

As robots become more complex, controlling them efficiently becomes increasingly challenging. A typical robot may contain multiple actuators such as wheel motors, robotic arm joints, grippers, or leg mechanisms. Each actuator may require different control modes such as position, velocity, or effort (torque) control.

ROS 2 Control is a framework that provides a standardized interface between ROS 2 applications and robot hardware. It abstracts the hardware details and allows the same controllers to operate both on simulated robots and on real physical robots with minimal changes.

Learning Objectives

By the end of this module, you will be able to:

- Understand the architecture of the ROS 2 Control framework.
- Configure a robot using the `<ros2_control>` tag in a URDF.
- Write a controller configuration using YAML.
- Integrate Gazebo with ROS 2 Control using the `gz_ros2_control` plugin.
- Launch and manage controllers using `controller_manager`.
- Bridge simulation topics between Gazebo and ROS 2 using `ros_gz_bridge`.

Refer to [Reference](#) for reference.

Why do we need ROS2 Controllers ?

Consider the line following bot that you made in previous task. There you used the differential plugin that subscribes to `/cmd_vel` topic, it receives from CLI (or through your teleop key) and then itself calculates the required velocities of wheels. So all calculations are handled by the plugin. Although it is useful, it reduces our control over the actuators. So the versatility of our bot gets marginalised.

Another reason is that, it works for a simple robot, but becomes increasingly difficult to manage as the robot grows in complexity. For e.g. there is no plugin that can control a quadruped bot's joints and we have to manually calculate the joint angles for motion. Also some of these motion plugins cannot be applied over real hardware.

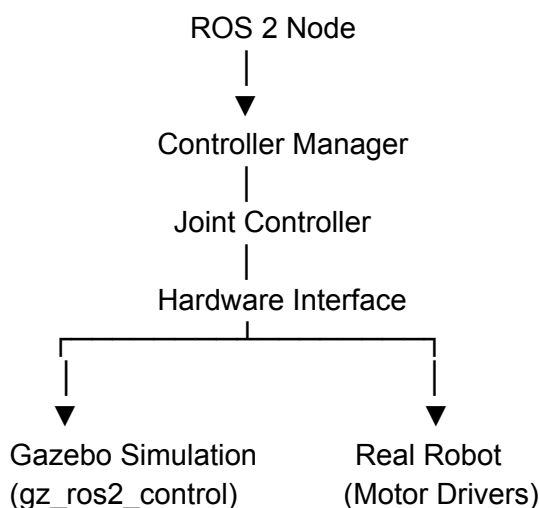
ROS 2 Control addresses these challenges by providing:

- A standard interface for all robot hardware.
- Reusable controllers for common control tasks.
- Support for position, velocity, and effort control.

- A unified framework for both simulation and real robots.
- Easy integration with Gazebo and RViz.
- Simplified controller management through ROS services and configuration files.

Simulation vs Real Robot

One of the biggest advantages of ROS 2 Control is that the same controller can be used in both simulation and on physical hardware. Only the hardware interface changes. The controller and the rest of the ROS 2 application remain the same, significantly reducing development and testing effort.



Components of ROS 2 Control

ROS 2 Node

A ROS node is responsible for generating commands for the robot. These commands may originate from:

- Teleoperation using a keyboard or joystick
- Navigation stack
- Motion planning algorithms
- Autonomous control software
- User-written programs

For example, a mobile robot receives velocity commands on the `/cmd_vel` topic, while a robotic arm may receive desired joint positions through trajectory messages. The ROS node does not communicate directly with the motors. Instead, it sends commands to the appropriate controller.

Controller Manager

The Controller Manager is the core component of the ROS 2 Control framework. Its responsibilities include:

- Loading controllers
- Starting and stopping controllers
- Switching between controllers
- Managing communication between controllers and hardware
- Updating controllers at a fixed control frequency

Only one Controller Manager exists for each robot. Without the Controller Manager, controllers cannot be loaded or executed.

Controllers

Controllers convert high-level commands into actuator commands that the hardware can understand. Different robots require different types of controllers. Some commonly used controllers include:

Controller	Purpose
Joint State Broadcaster	Publishes joint positions, velocities, and efforts
Diff Drive Controller	Controls differential drive mobile robots (used in IGVC)
Joint Trajectory Controller	Controls robotic arms using trajectories (used in arm)
Forward Position Controller	Sends desired joint positions (used in our quadruped)
Forward Velocity Controller	Sends desired joint velocities
Forward Effort Controller	Sends desired joint torques

Controllers operate continuously during robot execution and receive commands from ROS topics while monitoring the robot state.

Hardware Interface

The Hardware Interface acts as the communication layer between controllers and the robot. It performs two primary functions:

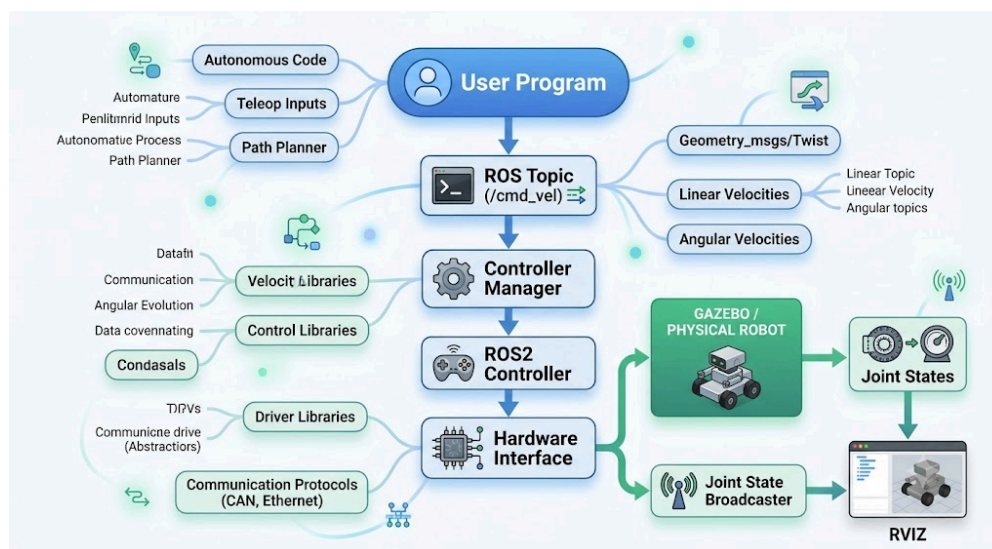
- Reads sensor and joint information from the robot.
- Sends actuator commands generated by the controllers.

This abstraction allows controllers to remain independent of the underlying hardware. For simulation, the Hardware Interface is provided by `gz_ros2_control`. For a real robot, it is implemented using hardware drivers that communicate with motor controllers, encoders, and sensors.

Only the hardware interface changes. The controller and the rest of the ROS 2 application remain the same, significantly reducing development and testing effort.

How Commands Flow Through ROS 2 Control

The complete execution flow is illustrated below:



Setting up the controllers

<ros2_control>

Refer to the [URDF](#) to understand how the control tags are added.

```
<ros2_control name="GazeboSystem" type="system">

<hardware>
| <plugin>ign_ros2_control/IgnitionSystem</plugin>
</hardware>

<joint name="left wheel joint">
| <command_interface name="velocity"/>
| <state_interface name="position"/>
| <state_interface name="velocity"/>
</joint>

<joint name="right wheel joint">
| <command_interface name="velocity"/>
| <state_interface name="position"/>
| <state_interface name="velocity"/>
</joint>

</ros2_control>
```

The `<ros2_control>` tag defines how the ROS 2 Control framework interacts with the robot. It specifies the hardware interface, the joints that can be controlled, and the command and state interfaces associated with each joint.

1. The `<ros2_control>` Tag

```
<ros2_control name="IgnitionSystem" type="system">
```

This is the root element of the ROS 2 Control configuration.

Parameters

name: `name="IgnitionSystem"`

This assigns a name to the hardware system managed by ROS 2 Control. Although the name can be chosen freely, it is good practice to use a meaningful name such as `IgnitionSystem`, `GazeboSystem`, or `MyRobotHardware`.

type: `type="system"`

The type specifies what kind of hardware component is being described.

Type	Description
actuator	Single actuator or motor
sensor	Sensor-only hardware
system	Complete robot with multiple joints

2. Hardware Plugin

Inside the `<ros2_control>` block, we define the hardware interface.

```
XML
<hardware>
  <plugin>ign_ros2_control/IgnitionSystem</plugin>
</hardware>
```

This plugin acts as the communication layer between Gazebo and ROS 2 Control. Instead of sending commands directly to Gazebo, controllers communicate with this plugin, which then applies the commands to the simulated robot. In a real robot, this plugin would be replaced by a hardware driver that communicates with motor controllers and sensors.

3. Joint Configuration

Each joint that should be controlled by ROS 2 Control must be declared individually. For example:

```
XML
<joint name="left_wheel_joint">
  <!-- Joint configuration goes here -->
</joint>
```

The joint name must exactly match the corresponding joint defined earlier in the URDF. If the names do not match, ROS 2 Control will fail to control the joint.

4. Command Interfaces

Each controllable joint specifies the type of command it accepts. Example:

```
XML
<command_interface name="velocity">
  <param name="min">-10</param>
  <param name="max">10</param>
</command_interface>
```

Here, the rear wheel receives velocity commands. The controller can command wheel speeds between -10 rad/s and 10 rad/s. Velocity control is commonly used for drive wheels because the robot is controlled by specifying wheel speeds.

Common command interfaces

Common command interfaces include:

Command Interface	Purpose
position	Desired joint position or angle
velocity	Desired joint speed
effort	Desired torque or force

In this example, the wheel joints are controlled using velocity commands, which is the standard approach for differential-drive robots.

State Interfaces

The controller also needs feedback from the robot. This is provided through state interfaces.

XML

```
<state_interface name="position"/>
```

This interface reports the current angular position of the wheel.

XML

```
<state_interface name="velocity"/>
```

This interface reports the current rotational velocity of the wheel. Controllers use this feedback to monitor the robot's state and perform closed-loop control.

Gazebo ROS 2 Control Plugin

The Gazebo ROS 2 Control plugin connects the Gazebo simulator with the ROS 2 Control framework. It initializes the hardware interface, starts the `controller_manager`, loads the controller configuration from a YAML file, and enables communication between Gazebo and ROS 2.

```
<gazebo>
  <plugin filename="libign_ros2_control-system.so" name="ign_ros2_control::IgnitionROS2ControlPlugin">
    <parameters>${controllers_file}</parameters>
    <ros>
      <remapping>/diff_drive_controller/cmd_vel_unstamped:=/cmd_vel</remapping>
    </ros>
  </plugin>
</gazebo>
```

`<remapping>` is optional as it is required when you need to map a topic name to some other topic. For e.g. the `diff_drive_controller` needs `/cmd_vel_unstamped` topic, but teleop key publishes `/cmd_vel` topic; so both are mapped.

Controller Configuration Using YAML

Till now everything that is done is largely common for any type of bot i.e. setting of `ros2_controller`. The YAML configuration file is where the control behavior is defined. It specifies which controllers to load, the joints they control, and the parameters required for their operation. Thus, while the URDF and plugin remain largely unchanged, the YAML file varies depending on the controller being used.

YAML Configuration in ROS 2 Control

ROS 2 uses YAML (YAML Ain't Markup Language) files to store configuration parameters for nodes. In ROS 2 Control, the YAML file is used to configure the Controller Manager and all the controllers that it manages. A YAML file organizes information using keys, values, and indentation. Unlike XML, YAML does not use tags; instead, indentation defines the hierarchy of the configuration.

The following example shows the basic structure of a ROS 2 Control configuration file:

None

```
controller_manager:
  ros__parameters:
    update_rate: 100

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    diff_drive_controller:
      type: diff_drive_controller/DiffDriveController
```

Understanding the Structure

The hierarchy of the YAML file is essential for the Controller Manager to read the parameters correctly:

- **controller_manager:** This is the name of the ROS 2 node whose parameters are being configured.
- **ros__parameters:** This section contains all the parameters that will be loaded into the controller_manager node when it starts.
- **update_rate: 100:** Specifies the frequency (in Hz) at which the Controller Manager updates all active controllers.

Controller Declaration

Each controller is declared under `ros__parameters`. For example:

None

```
joint_state_broadcaster:
```

```
type: joint_state_broadcaster/JointStateBroadcaster
```

This declares a controller named `joint_state_broadcaster` and specifies the plugin that implements it. Similarly:

None

```
diff_drive_controller:
  type: diff_drive_controller/DiffDriveController
```

This declares a Differential Drive Controller. Only the controller name and plugin type are declared here. The detailed configuration for each controller is provided later in separate sections of the YAML file.

Check the [YAML](#) to see how the `diff_drive_controller` is defined.

Note:- Don't forget to add the yaml file name in your CMakeLists.txt/Setup.py file.

The `joint_state_broadcaster` is common to almost all ROS 2 Control applications. Its purpose is to publish the current joint states (position, velocity, and effort) to the `/joint_states` topic, allowing tools such as RViz and other ROS nodes to monitor the robot's state. The second controller declaration depends on the application and determines how the robot is controlled. For example:

Controller Type	Application
<code>diff_drive_controller/DiffDriveController</code>	Differential drive robots
<code>ackermann_steering_controller/AckermannSteeringController</code>	Ackermann steering vehicles
<code>forward_position_controller/ForwardCommandController</code>	Position control
<code>forward_velocity_controller/ForwardCommandController</code>	Velocity control
<code>joint_trajectory_controller/JointTrajectoryController</code>	Robotic arms and manipulators

ROS–Gazebo Bridge

Gazebo and ROS 2 use different communication systems. ROS 2 communicates using DDS, while Gazebo uses its own Gazebo Transport middleware. Since these systems are not directly compatible, a bridge is required to exchange data between them. The ROS–Gazebo Bridge (`ros_gz_bridge` or `ros_ign_bridge`) translates messages between Gazebo and ROS 2, allowing ROS nodes to interact with the simulated robot. Without the bridge, topics published in Gazebo would not be accessible in ROS 2, and ROS 2 commands would not reach the simulator.

The bridge converts messages between the two communication systems in both directions.

Commonly Bridged Topics

Topic	Purpose
<code>/clock</code>	Simulation time
<code>/cmd_vel</code>	Robot velocity commands
<code>/scan</code>	LiDAR data
<code>/camera/image</code>	Camera images
<code>/imu</code>	IMU measurements
<code>/odom</code>	Robot odometry

Example

The following bridge exposes the simulation clock and LiDAR data to ROS 2:

```
Shell
ros2 run ros_gz_bridge parameter_bridge \
/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock \
/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan
```

Integrating Controllers and Bridges in the Launch File

The final step is to integrate all the components using a ROS 2 launch file. The launch file automates the startup of the complete simulation environment by launching Gazebo, publishing the robot description, spawning the robot, loading the controllers, and starting the ROS–Gazebo bridge.

Refer to this to understand how the controllers and bridges are preloaded in the launch file.

Then launch your launch file with `ros2 launch <package_name> <launch_file_name>`

On successful loading of the controllers you would see something like this:-

```
[spawner-5] [INFO] [1782567771.518485491] [spawner_joint_state_broadcaster]: Loaded joint_state_broadcaster
[ign gazebo-1] [INFO] [1782567771.518975998] [controller_manager]: Configuring controller 'joint_state_broadcaster'
[ign gazebo-1] [INFO] [1782567771.519045277] [joint_state_broadcaster]: 'joints' or 'interfaces' parameter is empty. All available state interfaces will be published
[spawner-5] [INFO] [1782567771.549311650] [spawner_joint_state_broadcaster]: Configured and activated joint_state_broadcaster
[INFO] [spawner-5]: process has finished cleanly [pid 94032]
[INFO] [spawner-6]: process started with pid [94192]
[ign gazebo-1] [INFO] [1782567771.940418104] [controller_manager]: Loading controller 'diff_drive_controller'
[spawner-6] [INFO] [1782567771.958785473] [spawner_diff_drive_controller]: Loaded diff_drive_controller
[ign gazebo-1] [INFO] [1782567771.960016829] [controller_manager]: Configuring controller 'diff_drive_controller'
[spawner-6] [INFO] [1782567771.990106756] [spawner_diff_drive_controller]: Configured and activated diff_drive_controller
[INFO] [spawner-6]: process has finished cleanly [pid 94192]
```

Exploring the Flexibility of ROS 2 Control

This demonstrates the ability to switch between different controllers at runtime without modifying the robot description, restarting Gazebo, or respawning the robot. Follow these steps:

1. **Integrate your robot with the ROS 2 Control framework.**
2. **Configure the following controllers in your YAML file:**
 - joint_state_broadcaster
 - diff_drive_controller
 - forward_velocity_controller
3. **In the launch file, load all three controllers**, but keep the Diff Drive Controller active and the Forward Velocity Controller inactive.
4. **Verify the controller status:**

Shell

```
ros2 control list_controllers
```

The output should show the Diff Drive Controller as active and the Forward Velocity Controller as inactive.

5. **Control the robot using the Diff Drive Controller:**

Shell

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.5}}" -r 10
```

Observe the robot's motion and stop the publisher (Ctrl+C) without closing Gazebo.

6. Switch the active controller:

Shell

```
ros2 control switch_controllers --deactivate diff_drive_controller  
--activate forward_velocity_controller
```

7. Verify that the controller has been switched:

Shell

```
ros2 control list_controllers
```

The output should now show the Forward Velocity Controller as active and the Diff Drive Controller as inactive.

8. Control the robot using the Forward Velocity Controller:

Shell

```
ros2 topic pub /forward_velocity_controller/commands  
std_msgs/msg/Float64MultiArray "{data: [5.0, 8.0]}" -r 10
```

To stop the robot, publish zero wheel velocities:

Shell

```
ros2 topic pub --once /forward_velocity_controller/commands  
std_msgs/msg/Float64MultiArray "{data: [0.0, 0.0]}"
```

Objective: This task demonstrates one of the major advantages of ROS 2 Control: the ability to switch between different controllers at runtime without modifying the robot description, restarting Gazebo, or respawning the robot. The same robot hardware can be controlled using different control strategies simply by changing the active controller.

A practical application is an amphibious robot that travels on land and floats on water. ROS 2 Control allows the robot to switch between different motion controllers at runtime, enabling smooth transitions between locomotion modes without changing the robot hardware or restarting the software.

Mini Task:- Write a `ros2` node for velocity controller for moving your line following bot in an approximate square shape.